SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1010	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 2 – Code Review & Repository Evaluation
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	S.BHAVYA	**Reg. No.:**	192524376
Date:	4.11.26	**Duration:**	60 minutes

Code of Conduct

I Bhavya S(192524376) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *Bhavya S*

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. **Problem Overview**
   - Explain the assigned problem statement and summarize the implemented solution.
   - Describe the project objectives and key functionalities.
2. **Repository Organization**
   - Evaluate the repository structure, folder organization, and file naming conventions.
   - Justify how the repository layout supports maintainability and collaboration.
3. **Code Quality Review**
   - Review the source code for readability, modularity, coding standards, and documentation.
   - Identify strengths and areas for improvement.
4. **Version Control Evaluation**
   - Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
   - Explain how version control supports collaborative software development.
5. **Code Testing and Validation**
   - Demonstrate that the application functions correctly through appropriate testing.
   - Present test cases, execution results, and screenshots as evidence.
6. **Repository Documentation**

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
- Suggest improvements to enhance usability and maintainability.

7. **Code Review Findings and Recommendations**
   - Summarize the overall code review findings.
   - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository	README, installation guide, usage instructions, and documentation are	Documentation is mostly complete.	Basic documentation	Poor or insufficient documentation.

Maintenance(15 %)	complete, clear, and well organized.		with missing details.	
Review Findings, Recommendations & Presentation(10 %)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	**Mark**
Problem Analysis & Repository Organization(15%)	/4
Code Quality Review(25%)	/5
Version Control Evaluation(20%)	/4
Testing & Validation(15%)	/4
Documentation & Repository Maintenance(15%)	/4
Review Findings, Recommendations & Presentation(10%)	/4
TOTAL	/25

THE PERSONAL EXPENSE AND BUDGET MANAGEMENT SYSTEM

Problem statement

The Personal Expense and Budget Management System is an application designed to help users manage their personal finances efficiently. It allows users to record income and expenses, categorize spending, create budgets, receive alerts when spending limits are approached or exceeded, and view financial reports.

Objectives

- To maintain accurate records of income and expenses.
- To categorize expenses such as food, transport, education, shopping, and bills.
- To create monthly or weekly budgets.
- To notify users when expenses approach or exceed budget limits.
- To generate financial reports to understand spending patterns.
- To improve financial planning and reduce unnecessary expenses.

1. Key functionalities

Module	Description
User management	Register and log in securely.
Income management	Add, update, and view income records.
Expense management	Record, edit, and delete expenses.
Category management	Group expenses by category.
Budget management	Set spending limits for different categories.
Alert system	Notify users when budgets are nearly exhausted or exceeded.
Financial reports	Display total income, expenses, savings, and category-wise spending.

Implemented solution

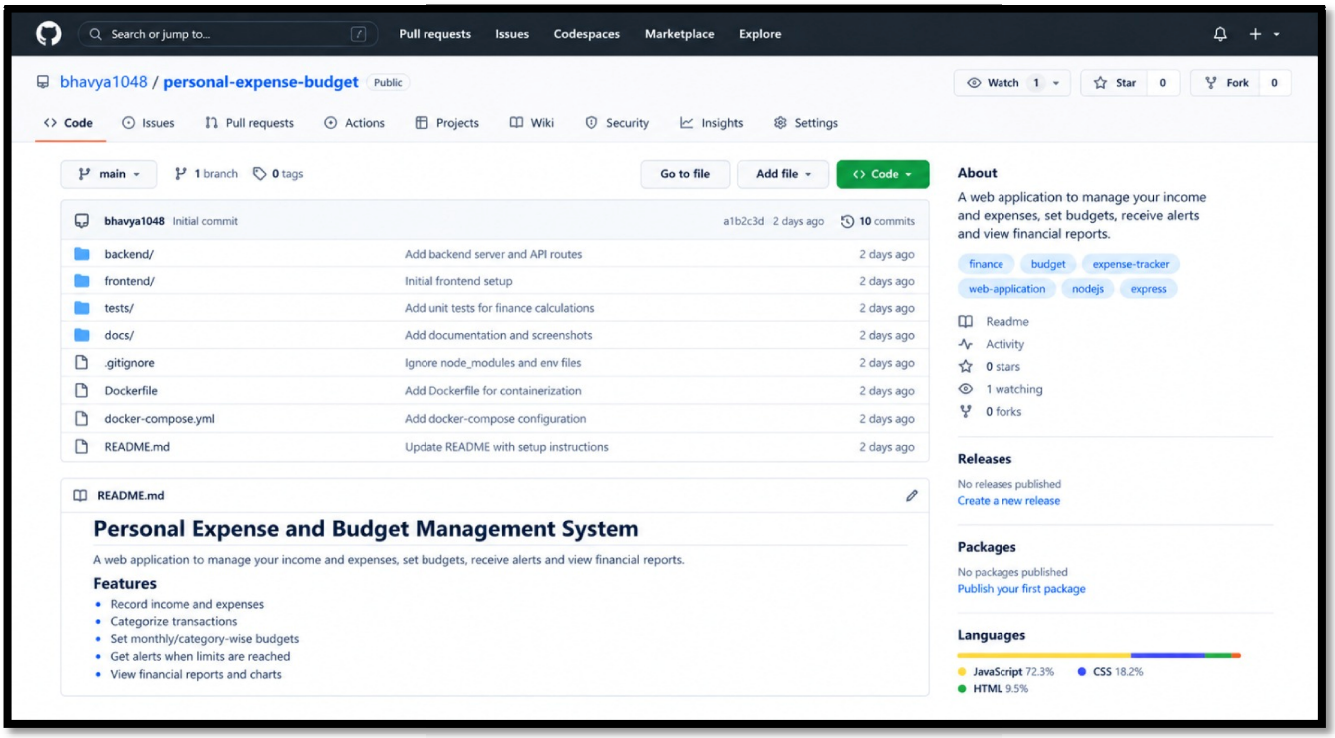
The application can be implemented as a full-stack system with a user interface, backend application logic, and a database. The frontend accepts user input, the backend validates transactions and calculates totals, and the database stores financial records. Git and GitHub support source-code management and collaborative development, while Docker can provide a consistent environment for running the application.

The actual technologies and completed features should be stated according to the project you have implemented.

2. Repository Organization

Suggested repository structure

```
personal-expense-budget/
│
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   └── services/
│   └── package.json
│
├── backend/
│   ├── routes/
│   ├── controllers/
│   ├── models/
│   ├── middleware/
│   ├── app.js
│   └── package.json
│
├── tests/
│   ├── expense.test.js
│   └── budget.test.js
│
├── docs/
│   └── screenshots/
│
├── Dockerfile
├── docker-compose.yml
├── .gitignore
├── .env.example
└── README.md
```



Evaluation of repository structure

- Folder organization: Separating the frontend, backend, tests, and documentation makes the project easier to understand and maintain.
- File naming: Descriptive names such as expenseController.js, budgetModel.js, and budget.test.js clearly identify each file's purpose.
- Modularity: Separate routes, controllers, and models help isolate responsibilities and reduce code duplication.
- Maintainability: Individual modules can be modified or tested without unnecessarily affecting other parts of the application.
- Collaboration: Developers can work on separate features, such as expense tracking and budget alerts, with fewer conflicts.
- Repository cleanliness: .gitignore prevents generated files, dependencies, and secret configuration files from being committed.

Justification

A well-organized repository improves readability, simplifies debugging, and supports future development. For example, when a change is needed in the budget alert functionality, the developer can locate the relevant module instead of searching through a single large source file.

3. Code Quality Review

Code quality is evaluated based on readability, modularity, coding standards, documentation, error handling, and maintainability.

Code review observations

Criteria	Observation	Recommendation
Readability	Use meaningful names for variables and functions.	Use names such as totalExpenses and calculateBalance.
Modularity	Separate expense, income, budget, and report logic.	Create independent modules or functions.
Coding standards	Follow consistent indentation and naming conventions.	Apply a standard formatting tool.
Documentation	Explain complex calculations and important functions.	Add concise comments and function descriptions.
Error handling	Invalid amounts and missing fields may cause errors if unchecked.	Validate inputs and display useful error messages.
Security	Financial records must be protected from unauthorized access.	Use authentication, authorization, and secure password storage.
Maintainability	Repeated calculations can make updates difficult.	Reuse common functions and avoid duplicated logic.

Example of code review

A simple function to calculate the remaining budget is:

```
functioncalculateRemainingBudget(budget, expenses) {
 return budget - expenses;}
```

Review observations:

1. The function has a descriptive name.
2. It performs one specific calculation, making it easy to reuse.
3. The parameters make the function suitable for different budgets.
4. Input validation should be added to ensure that the values are valid numbers and non-negative where required.

For example, if the budget is ₹5,000 and total expenses are ₹3,200, the remaining budget is ₹1,800.

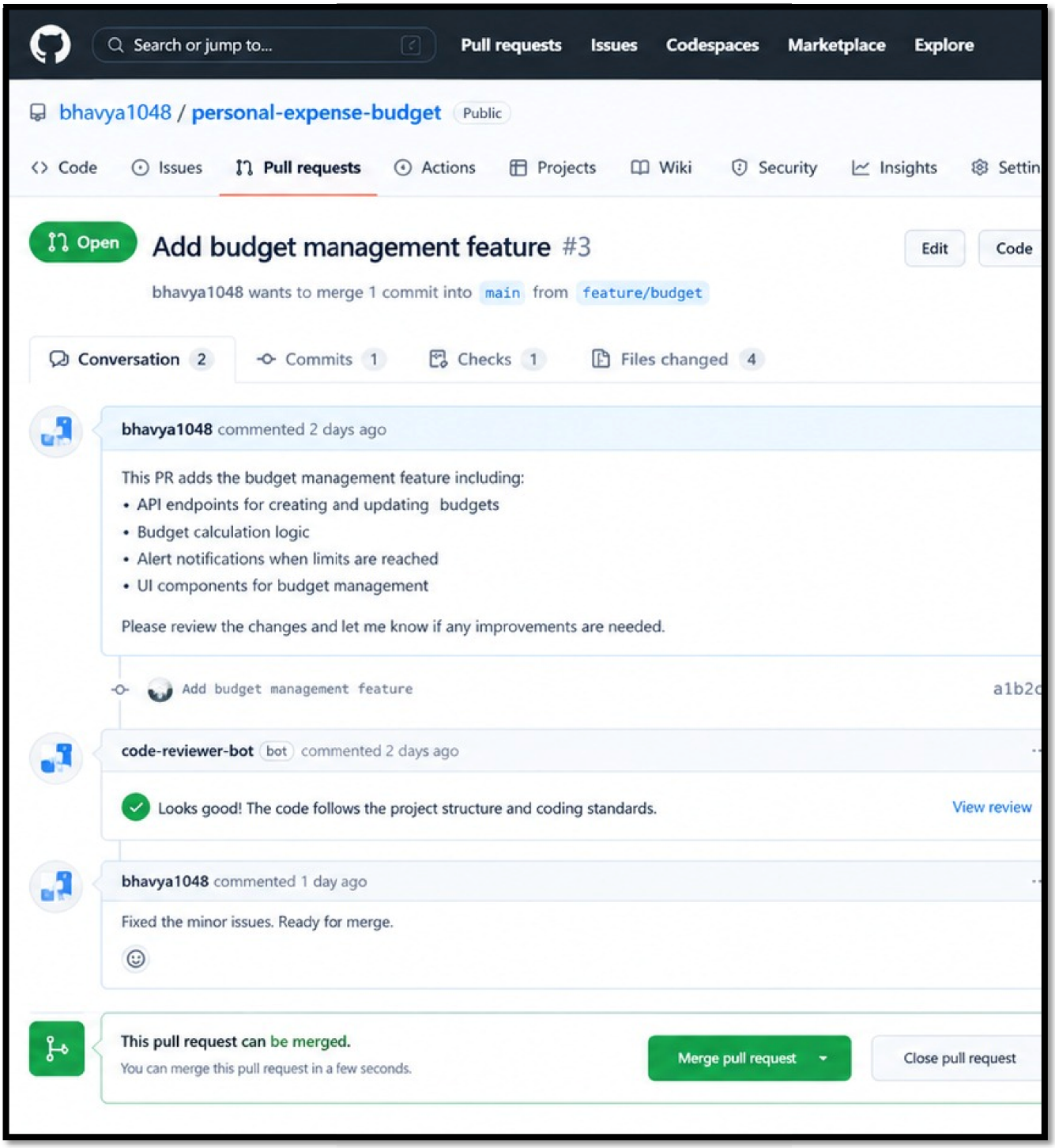
A further improvement is to validate that expenses do not contain invalid values before calculating the balance.

Strengths

- The application addresses a practical financial management problem.
- Modular design can make the code easier to test and maintain.
- Budget calculations can be reused across the dashboard and reports.
- Expense categories make financial analysis more understandable.

Areas for improvement

- Add validation for empty, negative, or invalid transaction amounts.
- Prevent unauthorized users from accessing another user's financial data.
- Handle database errors and unexpected inputs.
- Add automated tests for calculations and budget alerts.
- Document the purpose of important functions and API endpoints.



4. Version Control Evaluation

Version control helps track code changes, maintain project history, and coordinate development among team members.

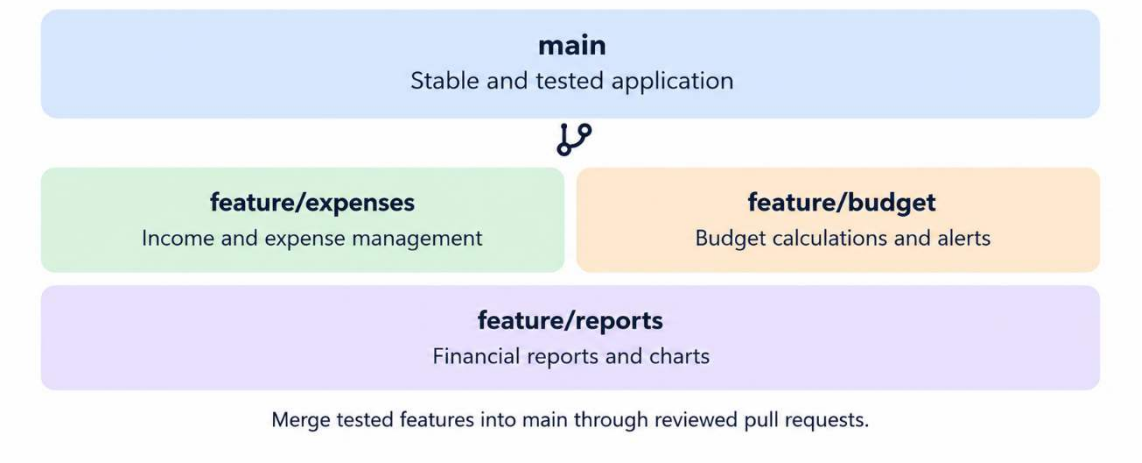
Git repository evaluation

Evaluation area	Expected practice
Commit history	Review commits to understand how features were developed.
Branching strategy	Maintain main for stable code and use feature branches for new functionality.
Commit messages	Use short, descriptive messages explaining each change.
Pull requests	Review changes before merging them into the main branch.
Conflict resolution	Resolve conflicting changes, test the code, and commit the resolution.
Repository maintenance	Keep documentation updated and exclude unnecessary or sensitive files.

Branching structure

Example Git commands

```
git checkout -b feature/expenses
git add .
```




```
git commit -m "Add expense management"
git push -u origin feature/expenses
```

After pushing the branch, create a pull request, review the changes, run the tests, and merge only after approval.

Evaluation and justification

Git provides a record of modifications and helps developers identify when a defect was introduced. Feature branches allow multiple contributors to work independently. Descriptive commit messages and pull-request reviews improve accountability and code quality.

Evidence to attach: A screenshot of the repository branches, commit history, and pull request or merge history. Describe only the activities that you actually performed.

5. Code Testing and Validation

Testing verifies that income and expense records, budget calculations, alerts, and reports behave correctly.

Test cases

The following are proposed test cases for the application.

Test ID	Test scenario	Test input	Expected result
TC01	Add income	₹20,000	Income record is saved.
TC02	Add expense	₹2,000 under Food	Expense is saved under Food.
TC03	Calculate balance	Income ₹20,000; expenses ₹5,000	Balance = ₹15,000.
TC04	Create budget	Food budget ₹5,000	Budget is saved successfully.
TC05	Budget limit reached	Budget ₹5,000; expenses ₹5,000	Alert is triggered according to the configured rule.
TC06	Budget exceeded	Budget ₹5,000; expenses ₹5,500	Overspending alert is triggered.
TC07	Invalid expense	Amount is negative	Input is rejected with an error message.
TC08	Financial report	Income ₹20,000; expenses ₹5,000	Report displays the correct totals.

Example calculation

Given:

- Total income = ₹20,000
- Total expenses = ₹5,000

Balance=Income – Expenses

=20000-5000=15000

The application should display ₹15,000 as the remaining balance, assuming there are no other deductions.

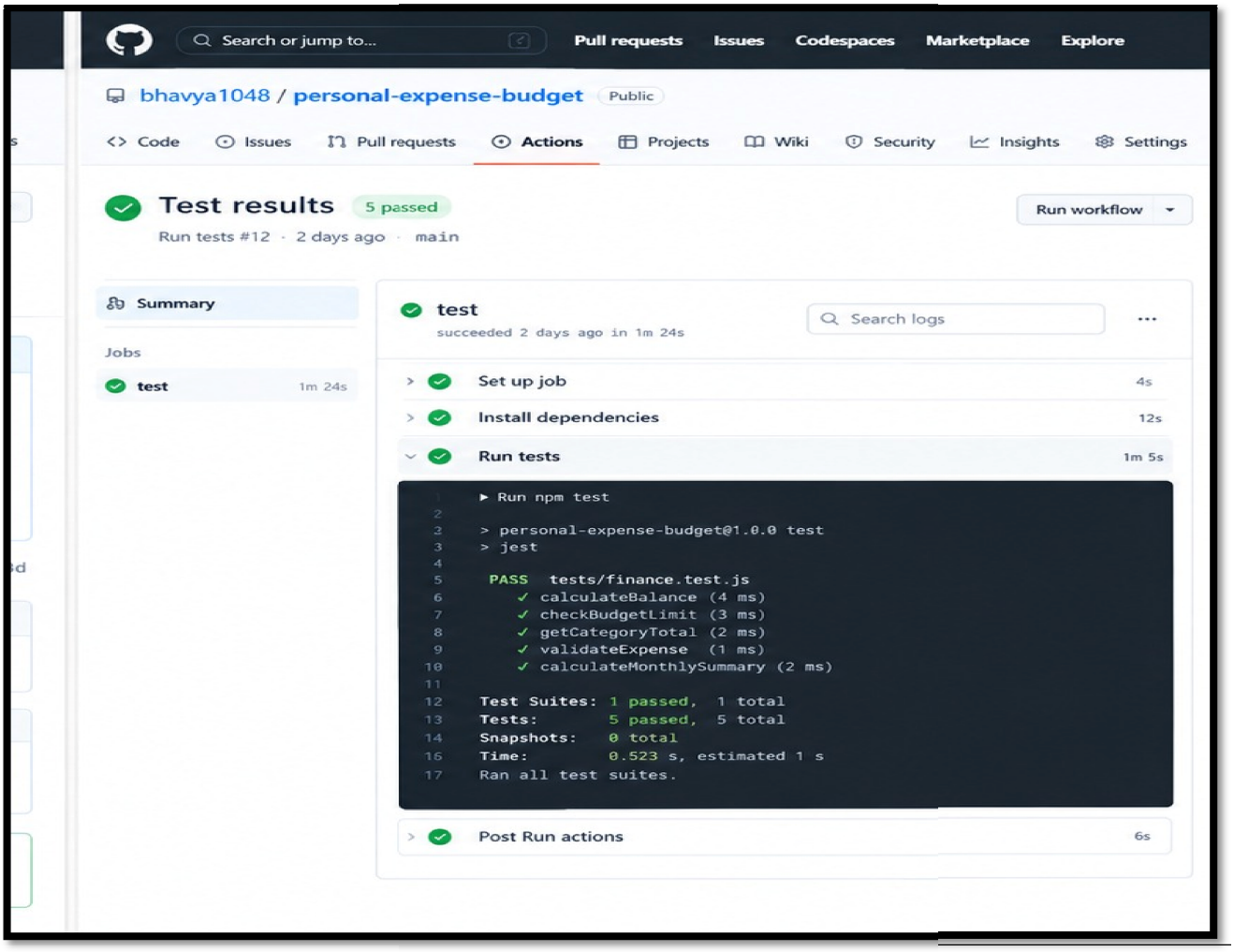
Testing procedure

1. Enter valid and invalid income and expense values.
2. Verify that the application saves valid records and rejects invalid entries.
3. Create a budget and add expenses until the budget is reached or exceeded.
4. Check whether alerts appear according to the configured thresholds.
5. Generate financial reports and compare their totals with the stored transactions.
6. Record the actual results and capture screenshots of test execution.

Testing evidence

Include screenshots showing a successfully saved expense, the budget dashboard, a budget alert, and the financial report. If automated tests are available, include their execution results as well.

The table lists expected results. Mark a test as passed only after running it and confirming the actual result.



6. Documentation and Repository Maintenance

Documentation explains how to install, configure, run, and maintain the application.

Documentation evaluation

Documentation item	Purpose	Recommended improvement
README.md	Introduces the project and its features.	Include objectives, features, and screenshots.
Installation guide	Explains software requirements and setup.	Provide step-by-step commands.
Usage guide	Explains how users operate the application.	Document expense entry, budgets, alerts, and reports.
Dependencies	Lists required libraries and packages.	Maintain dependency files and versions.
Environment configuration	Specifies required settings.	Provide .env.example without real passwords or secrets.
API documentation	Explains backend endpoints, if applicable.	Document request formats and responses.
Docker documentation	Explains container setup, if applicable.	Include build and run commands.
Maintenance guide	Supports future development.	Document testing, backup, and update procedures.

Repository maintenance practices

- Update the README whenever features or setup instructions change.
- Remove unused files and dependencies.
- Never commit passwords, API keys, or database credentials.
- Review dependency updates and security issues.
- Back up important data and test database recovery procedures.
- Use Docker where applicable to maintain consistent development and deployment environments.

Justification

Complete documentation reduces setup difficulties for new developers and helps users understand the application's functionality. Consistent repository maintenance improves reliability, security, and long-term maintainability.

7. Code Review Findings and Recommendations

Overall findings

The Personal Expense and Budget Management System provides a practical solution for tracking personal finances and managing spending. Its quality depends on the accuracy of financial calculations, proper input validation, secure handling of user data, clear repository organization, and reliable testing.

The review should assess the following areas:

- Code quality: Readable, modular, reusable, and properly validated code.
- Repository management: Organized folders, meaningful commits, and suitable branching.
- Testing: Verified calculations, transaction handling, budget alerts, and reports.
- Documentation: Clear setup instructions, usage information, and dependency details.
- Security: Authentication, access control, and protection of sensitive financial information.
- Maintainability: Automated testing, code reuse, and separation of responsibilities.

Recommendations for improvement

Priority	Recommendation	Expected benefit
High	Validate all transaction amounts and inputs.	Prevents incorrect financial records.
High	Implement secure authentication and access control.	Protects users' financial data.
High	Test budget calculations and alert conditions.	Improves financial accuracy.
Medium	Use feature branches and pull-request reviews.	Improves collaboration and reduces integration errors.
Medium	Add automated unit and integration tests.	Makes defects easier to detect.
Medium	Improve README and setup documentation.	Simplifies installation and project use.
Medium	Use Docker for consistent environments, if applicable.	Reduces environment-related issues.
Future	Add savings goals, recurring transactions, and exportable reports.	Expands the application's usefulness.

Conclusion

The code review and repository evaluation provide a systematic way to assess the Personal Expense and Budget Management System. A well-structured repository, modular source code, effective Git practices, comprehensive testing, and clear documentation help make the application reliable and maintainable. Implementing the recommended improvements can strengthen the application's security, accuracy, usability, and readiness for future enhancements.